Data Pipeline using Azure Data Factory, Databricks & Unity Catalog

BootCamp_Project-4

Student Name: Bhaskar Karri

Program: Azure Data Engineering

Table of Contents

BootCamp_Project-4 ..... **1**
1. Project Overview **2**
2. Problem Statement **3**
Objective 3
3. Solution Architecture **3**
4. Tools & Technologies **4**
5. Data Sources **4**
6. Data Schema **4**
7. Implementation Steps **5**
Step 1: Unity Catalog Setup 5
Step 2: Data Ingestion (Bronze Layer) 5
Output: 6
Step 3: Data Transformation (Silver Layer) 6
Transformations: 6
Output: 7
Step 4: Data Aggregation (Gold Layer) 8
Aggregations: 8
Output: 10
Step 5: Visualization using Plotly 11
Visuals: 11
Step 6: Orchestration using Azure Data Factory 12
Pipeline Flow: 12
Step 7: Pipeline Scheduling 12
8. Challenges Faced **12**
9. Solutions Implemented **13**
10. Key Learnings **13**
11. Final Conclusion **13**

1. Project Overview

This project demonstrates the design and implementation of a modern data engineering pipeline using Azure cloud services. The solution ingests raw data from external sources (GitHub), processes it using Azure Databricks, and orchestrates workflows using Azure Data Factory (ADF).

A Medallion Architecture (Bronze → Silver → Gold) is implemented to ensure data quality, scalability, and performance. The final curated data is used to generate interactive visualizations using Plotly.

2. Problem Statement

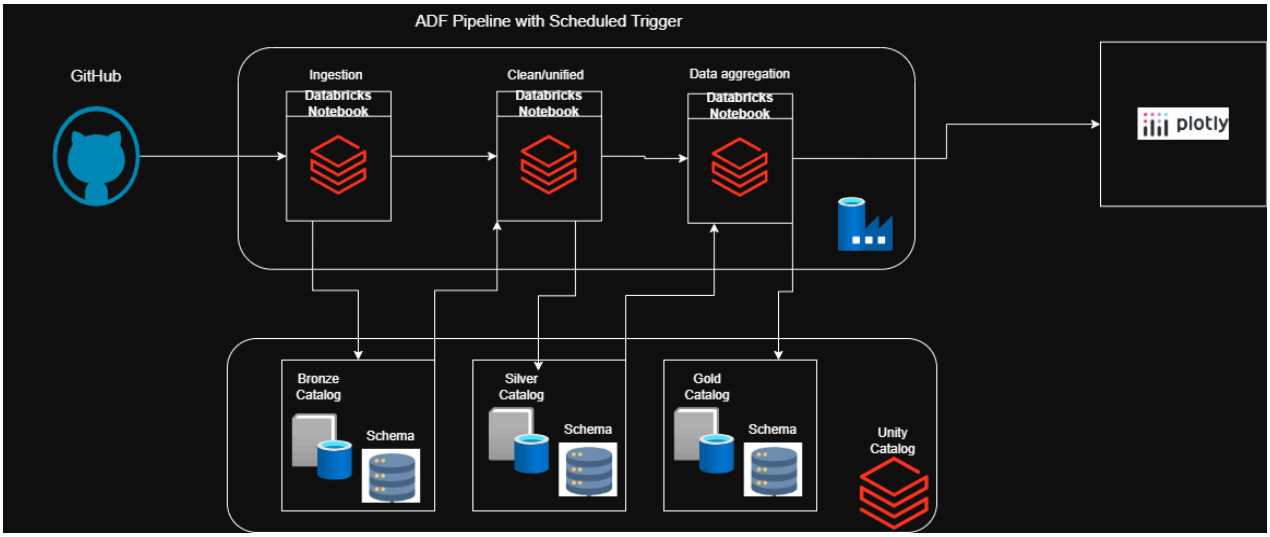
- Handling data from multiple external sources
- Ensuring data quality and consistency
- Performing scalable transformations
- Automating data workflows
- Generating meaningful insights

Objective

To build a pipeline that:

- Ingests data from GitHub
- Cleans and transforms it efficiently
- Stores it in a structured format
- Enables automated orchestration
- Provides visual insights

3. Solution Architecture



4. Tools & Technologies

Tool	Purpose
Azure Data Factory	Pipeline orchestration & scheduling
Azure Databricks	Data ingestion & transformation
Unity Catalog	Data governance & storage
Delta Lake	Reliable and scalable data format
GitHub	Source of data
PySpark	Data processing
Plotly	Interactive visualization

5. Data Sources

The project uses three datasets from GitHub:

- 2019 Data
- 2020 Data
- 2021 Data

https://raw.githubusercontent.com/MicrosoftLearning/mslearn-databricks/main/data/2019.csv

https://raw.githubusercontent.com/MicrosoftLearning/mslearn-databricks/main/data/2020.csv

https://raw.githubusercontent.com/MicrosoftLearning/mslearn-databricks/main/data/2021.csv

6. Data Schema

The dataset includes the following columns:

- EmployeeId
- Line_Number

- Audit_Date
- Employee_Name
- Employee_Email
- Store_Address
- Shift_Id
- Sales_Amount
- Profit

7. Implementation Steps

Step 1: Unity Catalog Setup

- Created Metastore
- Created Catalog
- Created Schema
- Created Volume

This serves as the centralized storage layer.

```
%sql

CREATE VOLUME IF NOT EXISTS
bronzeIngestion.bc04_br_schema.customers_volume;

CREATE VOLUME IF NOT EXISTS
silverCleaning.bc04_sl_schema.customers_volume;

CREATE VOLUME IF NOT EXISTS
goldAggregated.bc04_go_schema.customers_volume;
```

Step 2: Data Ingestion (Bronze Layer)

- Data fetched from GitHub using Databricks Notebook

- Since files had no headers:
    - Schema was manually defined
- Data was converted into Spark DataFrame
- Stored in Delta format

Output:

```
base_path =
"/Volumes/bronzeIngestion/bc04_br_schema/customers_volume/customers_data/
"
```

/Volumes/.../customers_data/sales_2019

/Volumes/.../customers_data/sales_2020

/Volumes/.../customers_data/sales_2021

Purpose: Store raw structured data

Step 3: Data Transformation (Silver Layer)

- Read data from Bronze layer
- Merged all yearly datasets
- Applied cleaning transformations:

Transformations:

- Removed duplicates
- Filtered null values
- Trimmed string columns
- Casted data types
- Added derived columns:

- Year
- Audit_Year
- Audit_Month

```
4 days ago (1s)                                                    3

#Data Cleaning and Transformation
from pyspark.sql.functions import *

# Filtering Null rows
df_clean = df_all.dropDuplicates() \
    .filter(col("EmployeeId").isNotNull()) \
    .filter(col("Sales_Amount").isNotNull()) \
    .filter(col("Profit").isNotNull())

# Triming string columns
df_clean = df_clean \
    .withColumn("Employee_Name", trim(col("Employee_Name"))) \
    .withColumn("Employee_Email", trim(col("Employee_Email"))) \
    .withColumn("Store_Address", trim(col("Store_Address")))
df_clean.show(5)
(2) Spark Jobs
```

```
from pyspark.sql import functions as F



df_clean = df_clean.withColumn("Year", col("Year")) \

    .withColumn("Audit_Year", year(col("Audit_Date"))) \

    .withColumn("Audit_Month", month(col("Audit_Date")))

df_clean.show(5)
```

Output:

4 days ago (1s) 6

```
df = spark.read.format("delta").load(silver_path + "sales_clean")
df.show(5)
df.printSchema()
```

(1) Spark Jobs

df: pyspark.sql.connect.dataframe.DataFrame = [EmployeeId: string, Line_Number: integer ... 10 more fields]

```
+----------+-----------+----------+---------------+--------------------+--------------------+--------+------------+--------+----+----------+-------
----+
|EmployeeId|Line_Number|Audit_Date|  Employee_Name|      Employee_Email|       Store_Address|Shift_Id|Sales_Amount|  Profit|Year|Audit_Year|Audit_M
onth|
+----------+-----------+----------+---------------+--------------------+--------------------+--------+------------+--------+----+----------+-------
----+
|   SO44666|          1|2019-10-16|    Mohamed Pal|mohamed0@adventur...|    Road-150 Red, 52|       1|     3578.27|286.2616|2019|      2019|
10|
|   SO45121|          1|2019-12-09|Victoria Bailey|victoria33@advent...|    Road-150 Red, 62|       1|     3578.27|286.2616|2019|      2019|
12|
|   SO45238|          1|2019-12-27|     Tasha Deng|tasha1@adventure-...|Mountain-100 Silv...|       1|     3399.99|271.9992|2019|      2019|
12|
|   SO45141|          1|2019-12-12|     Evan Perez|evan28@adventure-...|    Road-150 Red, 48|       1|     3578.27|286.2616|2019|      2019|
12|
|   SO45154|          1|2019-12-14|    Gary Alonso|gary19@adventure-...|    Road-150 Red, 52|       1|     3578.27|286.2616|2019|      2019|
12|
+----------+-----------+----------+---------------+--------------------+--------------------+--------+------------+--------+----+----------+-------
----+
```

```
/Volumes/silverCleaning/bc04_sl_schema/customers_volume/cleaned_customers
_data/
```

Purpose: Create clean and standardized dataset

Step 4: Data Aggregation (Gold Layer)

- Read cleaned data from Silver
- Created multiple aggregated datasets:

Aggregations:

- Sales by Employee

4 days ago (9s)

```python
# Sales by Employees
from pyspark.sql.functions import sum

df_emp = df.groupBy("EmployeeId", "Employee_Name") \
    .agg(
        sum("Sales_Amount").alias("Total_Sales"),
        sum("Profit").alias("Total_Profit")
    )
df_emp.show(5)
```

(2) Spark Jobs

df_emp: pyspark.sql.connect.dataframe.DataFrame = [EmployeeId: string

```
+----------+----------------+-----------+------------+
|EmployeeId|   Employee_Name|Total_Sales|Total_Profit|
+----------+----------------+-----------+------------+
|   SO44665| Fernando Nelson|   699.0982|     55.9279|
|   SO45118|Jonathon Navarro|    3578.27|    286.2616|
|   SO43811|      Harold Sai|    3399.99|    271.9992|
|   SO44166|     Kevin Adams|   699.0982|     55.9279|
|   SO44240|        Carol Xu|    3578.27|    286.2616|
+----------+----------------+-----------+------------+
```

- Sales by Store

4 days ago (1s) 3

```python
# Sales by Store

from pyspark.sql.functions import sum, round

df_store = df.groupBy("Store_Address") \
    .agg(
        round(sum("Sales_Amount"), 2).alias("Total_Sales"), #roundingup results
        round(sum("Profit"), 2).alias("Total_Profit")
    )

df_store.show(5)
```

- Yearly Sales Trend

```
4 days ago (1s)

# yearly sales trend

df_year = df.groupBy("Year") \
    .agg(
        sum("Sales_Amount").alias("Yearly_Sales"),
        sum("Profit").alias("Yearly_Profit")
    )
df_year.show(5)

(2) Spark Jobs

df_year: pyspark.sql.connect.dataframe.DataFrame = [Year:

+----+-------------------+-----------------+
|Year|       Yearly_Sales|    Yearly_Profit|
+----+-------------------+-----------------+
|2020|  6372462.141400008|509796.9952999997|
|2019| 3863120.2134000054|309049.6230999989|
|2021|1.069244007590387E7|855395.2285000242|
+----+-------------------+-----------------+
```

- Monthly Sales Trend

```
4 days ago (1s)

# monthly Sales trend
df_month = df.groupBy("Audit_Year", "Audit_Month") \
    .agg(
        sum("Sales_Amount").alias("Monthly_Sales"),
        sum("Profit").alias("Monthly_Profit")
    )
df_month.show(5)

(2) Spark Jobs

df_month: pyspark.sql.connect.dataframe.DataFrame = [Audit_Year: integ

+----------+-----------+------------------+-----------------+
|Audit_Year|Audit_Month|     Monthly_Sales|   Monthly_Profit|
+----------+-----------+------------------+-----------------+
|      2019|         10| 561681.4757999998|44934.51890000002|
|      2020|          6|485867.42400000023|38869.39590000002|
|      2021|          8|1034193.8199999765|82735.50559999974|
|      2021|          6|  862776.709999993|69022.13680000015|
|      2021|          5| 578691.3580999996|46295.31370000002|
+----------+-----------+------------------+-----------------+
```

Output:

```
gold_path =
"/Volumes/goldAggregated/bc04_go_schema/customers_volume/cleaned_customer
s_data/"
```

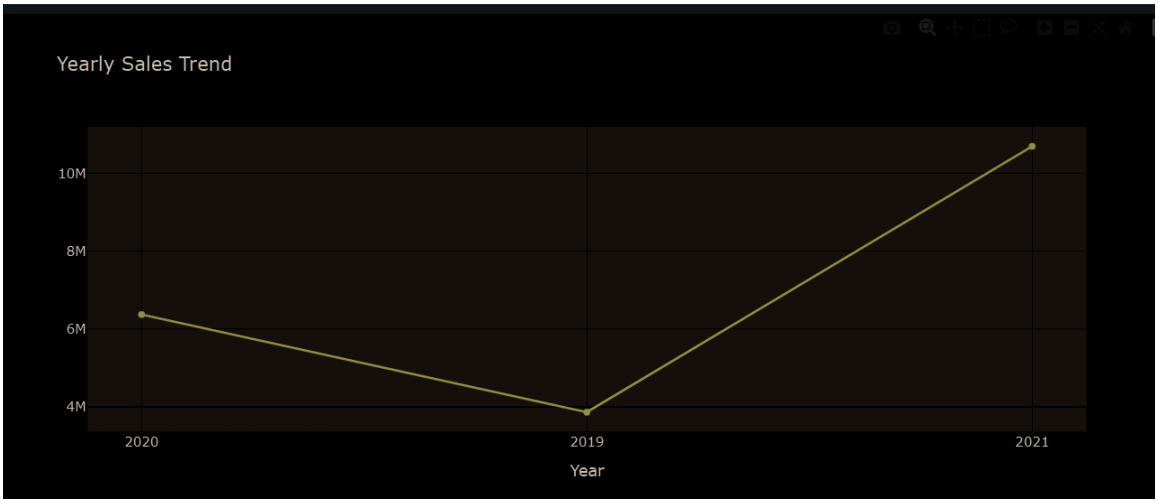
Purpose: Provide business-ready data

Step 5: Visualization using Plotly

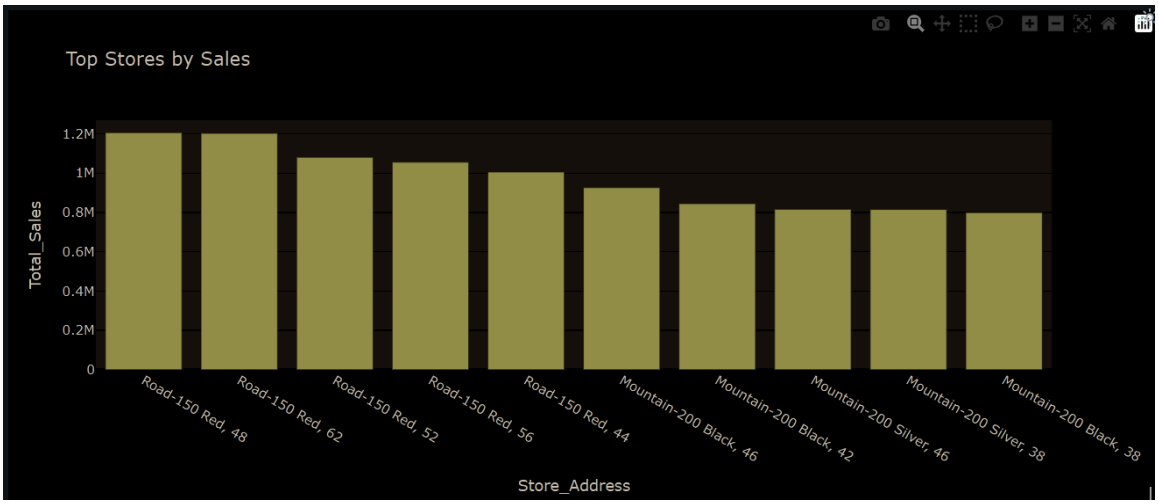
- Converted Spark DataFrames to Pandas
- Created interactive charts:

Visuals:

- Line chart (Yearly Sales Trend)



- Bar chart (Store Performance)

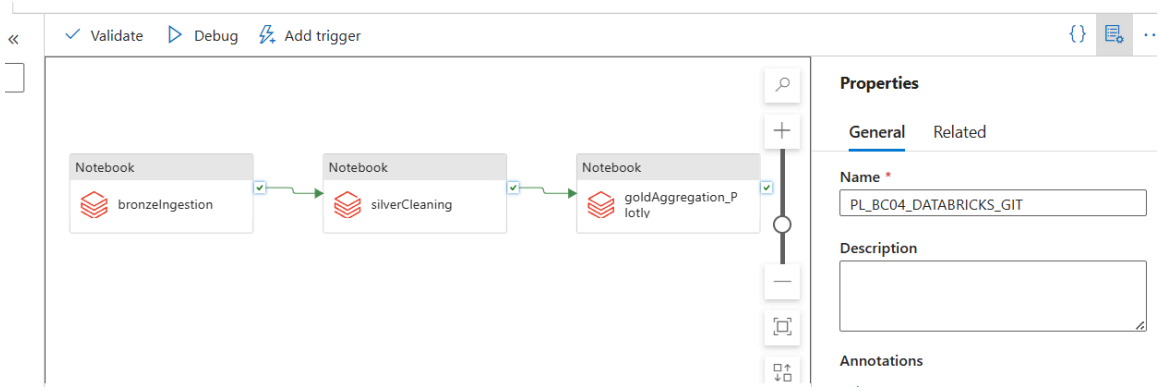


Step 6: Orchestration using Azure Data Factory

- Created ADF pipeline
- Added Databricks Notebook activities:

Pipeline Flow:

1. Ingestion Notebook
2. Silver Transformation Notebook
3. Gold Aggregation Notebook



Step 7: Pipeline Scheduling

- Configured trigger in ADF
- Scheduled pipeline (daily/hourly)

Enables automated data processing

8. Challenges Faced

- Reading data from GitHub in Spark (HTTPS limitation)
- Missing column headers in dataset
- Schema consistency across files
- Handling null and duplicate records

- Managing Unity Catalog paths

9. Solutions Implemented

- Used pandas for GitHub ingestion
- Defined schema manually
- Applied consistent transformations
- Used Delta format for reliability
- Modularized notebooks for orchestration

10. Key Learnings

- Medallion architecture improves data quality
- Databricks simplifies big data processing
- Unity Catalog centralizes governance
- ADF enables efficient orchestration
- Plotly enhances data storytelling

11. Final Conclusion

This project demonstrates a complete modern data engineering pipeline using:

- Azure Data Factory for orchestration
- Azure Databricks for transformation
- Unity Catalog for governance
- Delta Lake for storage
- Plotly for visualization